SHARE: Shaping Data Distribution at Edge for Communication-Efficient Hierarchical Federated Learning

Yongheng Deng[1], Feng Lyu[2]*, Ju Ren[2], Yongmin Zhang[2], Yuezhi Zhou[1], Yaoxue Zhang[1], Yuanyuan Yang[3]
[1]Department of Computer Science and Technology, BNRist, Tsinghua University, Beijing, China
[2]School of Computer Science and Engineering, Central South University, Changsha, China
[3]Department of Electrical and Computer Engineering, Stony Brook University, Stony Brook, USA
*Corresponding Author
{dyh19@mails., zhouyz@mail., zhangyx@}tsinghua.edu.cn, {fenglyu, renju, zhangyongmin}@csu.edu.cn,
yuanyuan.yang@stonybrook.edu

2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS) | 978-1-6654-4513-9/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICDCS51616.2021.00012

***Abstract*—Federated learning (FL) can enable distributed model training over mobile nodes without sharing privacy-sensitive raw data. However, to achieve efficient FL, one significant challenge is the prohibitive communication overhead to commit model updates since frequent cloud model aggregations are usually required to reach a target accuracy, especially when the data distributions at mobile nodes are imbalanced. With pilot experiments, it is verified that frequent cloud model aggregations can be avoided without performance degradation if model aggregations can be conducted at edge. To this end, we shed light on the hierarchical federated learning (HFL) framework, where a subset of distributed nodes are selected as edge aggregators to conduct edge aggregations. Particularly, under the HFL framework, we formulate a communication cost minimization (CCM) problem to minimize the communication cost raised by edge/cloud aggregations with making decisions on edge aggregator selection and distributed node association. Inspired by the insight that the potential of HFL lies in the data distribution at edge aggregators, we propose *SHARE*, i.e., SHaping dAta distRibution at Edge, to transform and solve the CCM problem. In SHARE, we divide the original problem into two sub-problems to minimize the per-round communication cost and mean Kullback–Leibler divergence of edge aggregator data, and devise two light-weight algorithms to solve them, respectively. Extensive experiments under various settings are carried out to corroborate the efficacy of *SHARE*.**

I. Introduction

Due to the high penetration of mobile devices and the advancement of sensing technology, a vast amount of data is continuously generated at edge [1], [2]. Such massive data is usually aggregated and stored in the cloud, together with machine learning algorithms, to enable multifarious intelligent services [3]–[5]. However, significant privacy leakage issue arises when delivering sensitive raw data to the cloud via network transmission, which, as the uppermost impetuses, pushes forward the emergence of federated learning (FL) [6]–[9]. Unlike the centralized learning paradigm, FL enables cooperative learning of a global model at distributed computing nodes using local raw data. Instead of sending raw data to the cloud, only the learned model updates are committed to the cloud server for aggregation. Then, the global model at the cloud server is updated and sent back to the distributed computing nodes for next-round iteration. In this way, the global model is learned without compromising user privacy.

In FL, efficient communication is crucial for model update aggregation and global model distribution [10], [11]. Particularly, to reach a target learning accuracy, sufficient rounds of cloud aggregation are required especially when the training data at distributed nodes are imbalanced (i.e., the number of samples for classes/labels are quite different), which has been verified by our pilot experiments. In this case, frequent model updates from distributed nodes are vital to optimize the global model. Contradictorily, the distributed nodes are usually constrained in communication/bandwidth resources, being prohibitive to communicate with the cloud frequently. Besides, as the learning model structure is becoming much complex (e.g., deep neural networks), the data volume of model updates will increase significantly, aggravating the communication overhead.

There have been some efforts in the literature to improve the communication efficiency in FL. For instance, compression schemes, such as sparsification [12], [13], quantization [14], and sketching [15], [16], are applied to compress model updates to reduce the size of messages communicated at each round. They can reduce the per-round communication cost but at the price of non-negligible performance degradation in accuracy and convergence speed, since the information of model updates cannot be ideally recovered [15]. Other works try to reduce the total number of either cloud aggregation rounds [17], [18] or model updates [11], [17], that are required to reach the target learning accuracy. For example, in the work [11], Wang *et al.* proposed a scheme to dynamically identify the irrelevant model updates and exclude them from being uploaded to save communication cost. Differently, in this paper, we aim at exploiting edge aggregation efficacy to avoid frequent cloud aggregations without learning performance degradation, which is a new direction to save communication cost and can be integrated with the aforementioned methods to collectively enhance the communication efficiency.

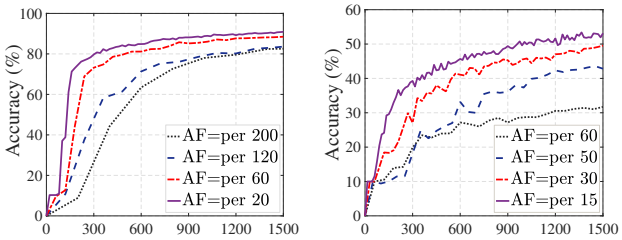
Specifically, to guarantee learning performance in typical cloud-based FL, distributed nodes are usually required to com-

2575-8411/21/$31.00 ©2021 IEEE
DOI 10.1109/ICDCS51616.2021.00012

mit model updates to the cloud aggregator frequently despite the significant communication overhead. However, according to our pilot experiments, we find that *frequent cloud aggregations can be avoided without performance degradation, if model aggregations can be conducted at edge*. The learning performance can be even enhanced when the *edge aggregation* is properly configured. This inspires us to investigate a hierarchical federated learning (HFL) framework, where a subset of distributed nodes can be selected as *edge aggregators*, to communicate with nearby nodes for edge model aggregations at low communication cost. After several rounds of edge aggregations, edge aggregators then commit edge model updates to the cloud for global aggregation. The merit is that frequent edge aggregations can be enabled with acceptable communication overhead, avoiding frequent communications to the cloud. Besides, we also disclose that *the potential of the HFL framework lies in the data distribution at edge aggregators*. Based on these insights, we propose *SHARE*, i.e., SHaping dAta distRibution at Edge, to solve our formulated *communication cost minimization* (**CCM**) problem, by making decisions on edge aggregator selection and distributed node association to minimize the total communication cost required to train the FL model.

In *SHARE*, we follow two optimization directions to transform and solve the original **CCM** problem. On the one hand, we focus on minimizing the communication cost of committing model updates to edge aggregators and the cloud aggregator, in order to *reduce the per-round communication cost*. On the other hand, we aim at shaping the data distribution at edge aggregators to make their data distributions being more balanced, which can *reduce the communication rounds required* to reach the target model accuracy. Since the two goals cannot be achieved simultaneously, we incorporate a parameter to tune the tradeoff and the original problem is transformed to the data distribution aware communication cost minimization (**DD-CCM**) problem. Due to the NP-hardness of the **DD-CCM** problem, we finally devise two light-weight algorithms to cooperatively solve the problem. In specific, given the set of edge aggregators, we first devise a Greedy-based nOde Association (*GoA*) algorithm to make decisions on node-to-edge associations. Based on *GoA*, we then devise a LOcal Search (*LoS*) algorithm to optimize the edge aggregator selection reversely. To evaluate the performance of *SHARE*, we build a practical HFL emulation platform by adopting the real-world network topologies and general learning tasks. Extensive experiments are conducted under various settings, and the results demonstrate that *SHARE* can enhance the learning accuracy and reduce the communication cost significantly, in comparison with two competitive benchmarks. We highlight our major contributions as follows.

- We uncover valuable insights with pilot experiments: 1) model aggregation frequency can affect the FL performance significantly; 2) using the HFL framework is promising in low-cost and high-performance learning; and 3) data distributions at edge aggregators can directly affect the performance of the HFL framework. They guide our route to minimize the communication cost under the HFL framework, which are also valuable for other researchers to optimize the desired FL system.
- Under the HFL framework, we are the first to investigate the communication cost minimization problem. Specifically, we formulate the **CCM** problem to make decisions on edge aggregator selection and node-to-edge association, which is of paramount importance to achieving a robust FL system and can give empirical guidelines to practical engineers.
- We propose *SHARE* to solve the original problem via: 1) associating distributed nodes with edge aggregators to minimize the per-round communication cost, and 2) shaping data distribution at edge aggregators to reduce the required communication rounds. Extensive experiment results verify that *SHARE* can be readily applied to various network topologies and different learning tasks with superior FL performance.



(a) MNIST (b) CIFAR-10

Fig. 1: Impact of model aggregation frequency.

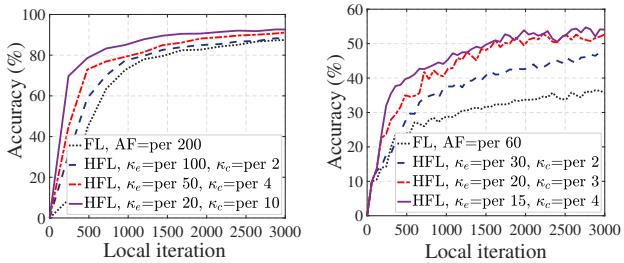
The remainder of this paper is organized as follows. We present our motivation in Section II. In Section III, we describe the system and define the research problem. Section IV elaborates on the design of *SHARE*. We conduct extensive experiments in Section V, and the related work is reviewed in Section VI. Finally, we conclude this paper and direct our future work in Section VII.

II. Motivation

In this section, we conduct pilot experiments to highlight the motivations of developing the HFL framework and the design of *SHARE*.

A. Model Aggregation Frequency Affects FL Performance Significantly

To achieve a satisfying global learning model, frequent model aggregations are usually required at the cloud where distributed computing nodes have to commit local model updates via long-distance transmissions, while the communication overhead is significant for both the resource-scarce mobile devices and the network. It is important to investigate how the model aggregation frequency can affect FL performance. To this end, we adopt the MNIST [19] and CIFAR-10 [20] datasets to carry out pilot experiments. In specific, we create a cloud-based FL system, where there are 30 distributed

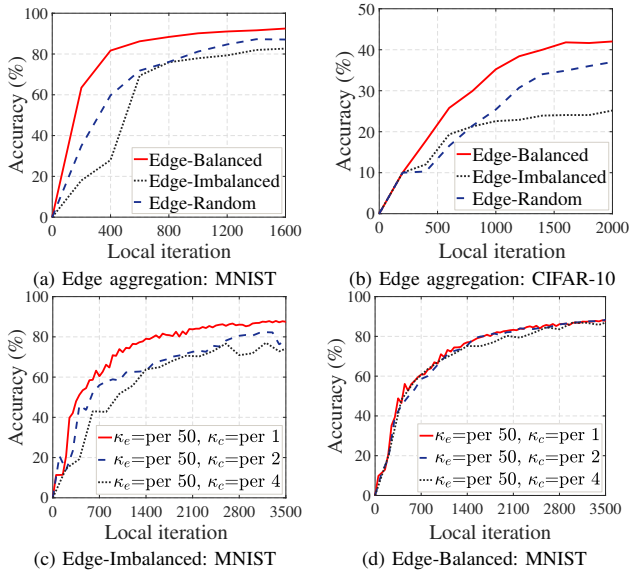


(a) MNIST (b) CIFAR-10

Fig. 2: Potential of using the HFL framework.

computing nodes and one cloud server. The two data sets have 10 classes of labeled data, and we divide each data set into 60 partitions after sorting the data in accordance with the class label. For each learning task, each computing node is randomly assigned with 2 partitions from 2 labels. Fig. 1 shows the accuracy on two datasets with different model aggregation frequency (AF), where AF = per 20 means one cloud aggregation is executed every 20 local iterations. We can observe that, for both learning tasks, a significant performance gap shows in model accuracy and convergence speed when enlarging the model aggregation frequency. Taking the MNIST dataset as an example, when AF = per 20, 300 iterations are sufficient to achieve 80% model accuracy, while 500, 1,000, and 1,200 iterations are required when AF = per 60, AF = per 120, and AF = per 200, respectively. In addition, conducting aggregations every 20 iterations also results in the highest final model accuracy. The experiment verifies that model aggregation frequency can affect FL performance significantly, when distributed nodes have imbalanced training data. To guarantee the learning performance, frequent global aggregations are a requisite, which on the other hand can raise prohibitive communication overhead to the network.

Potential of Using the HFL Framework. It would be tempting if some aggregations can be conducted at the network edge since the communication overhead to the edge is negligible compared with that to the cloud. Particularly, we investigate the usage of the HFL framework, where among distributed computing nodes, some nodes can be chosen as edge aggregators to conduct model aggregations for nearby nodes, and then the edge aggregators commit the edge model updates to the cloud for global aggregation. The underlying rationale is that frequent edge aggregations can be enabled with an acceptable communication overhead, which can alleviate the communication burden to the cloud. To this end, we create a HFL emulation system by adopting the similar experiment setting in terms of learning tasks and training data allocation. In comparison, among 30 distributed computing nodes, 3 nodes are chosen as additional edge aggregators, each randomly associated with 10 nodes for edge model aggregation. Fig. 2 shows the performance comparison between the HFL and cloud-based FL, where κ_e and κ_c are the aggregation frequency at edge and cloud, respectively. For instance, when κ_e = per 100, κ_c = per 2, it means that edge aggregation conducts every 100 local iterations, and one cloud



(a) Edge aggregation: MNIST (b) Edge aggregation: CIFAR-10

(c) Edge-Imbalanced: MNIST (d) Edge-Balanced: MNIST

Fig. 3: Trick of edge and cloud aggregation.

aggregation will be executed after every 2 edge aggregations. That said, one cloud aggregation and 2 edge aggregations will be executed every 200 local iterations. We can observe that for both learning tasks, the HFL framework outperforms cloud-based FL significantly in terms of model accuracy and convergence speed, regardless of the configuration of κ_e and κ_c. It demonstrates the potential of using the HFL framework, which can enhance the FL performance by re-locating frequent model aggregations at edge with acceptable communication cost.

B. Trick of Edge and Cloud Aggregation

We further investigate the trick of edge and cloud aggregation by adjusting the training data allocation at computing nodes. We consider the following three cases: 1) Edge-Balanced: assigning each edge aggregator 10 nodes with different classes of training data, i.e., each edge aggregator takes a uniformly sampled dataset from 10 classes; 2) Edge-Imbalanced: assigning each edge aggregator 10 nodes with the same class of training data as much as possible, where each edge aggregator has a rather skewed distribution of data; 3) Edge-Random: assigning each edge aggregator 10 nodes with randomly allocated data samples. Note that the data set of each edge aggregator is imbalanced, but is closer to the uniform distribution compared to that of the Edge-Imbalanced case. Fig. 3 (a) and (b) shows the performance comparison of the above three cases by learning with the MNIST dataset (κ_e = per 5, κ_c = per 40) and CIFAR-10 dataset (κ_e = per 50, κ_c = per 4), respectively. We can observe that the performance under the Edge-Balanced case can outperform that under the Edge-Imbalanced case significantly in terms of model accuracy and convergence speed. Moreover, it can be found that when the training data at edge aggregator is more balanced, i.e., closer to the uniform distribution, the final learning accuracy can be higher and fewer iterations are required to reach the

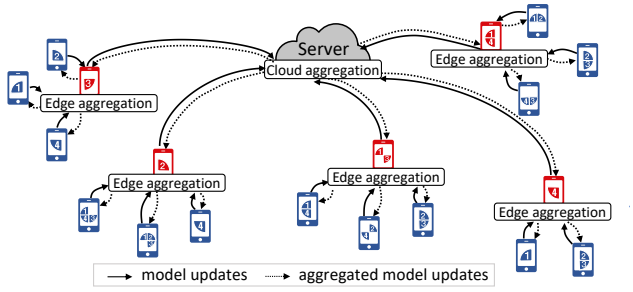


Fig. 4: An overview of the HFL framework.

desired accuracy. Therefore, we can conclude that in the HFL framework, when κ_e and κ_c are fixed, making the training data at edge aggregator closer to the uniform distribution can significantly enhance the federated learning performance and reduce the required communication rounds.

We then shed light on the cloud aggregation by varying κ_c when κ_e is fixed. Fig. 3 (c) and (d) show the HFL learning performance on the MNIST dataset under the Edge-Imbalanced and Edge-Balanced case, respectively. We can observe that, under the Edge-Imbalanced case, decreasing the cloud aggregation frequency (increasing κ_c) can lead to a significant performance degradation in terms of model accuracy and convergence speed. However, in the Edge-Balanced case, there is a negligible performance degradation. It means that if the training data at edge aggregator can be closer to the uniform distribution, frequent cloud aggregations can be avoided without performance degradation, which can further save the system communication cost.

C. Takeaways

With the pilot experiments, we can conclude that: 1) frequent model aggregations contribute to a superior FL model, while the communication overhead to the cloud is prohibitive in the cloud-based FL system; 2) the HFL framework is promising to support distributed learning in a cost-efficient manner, where frequent model aggregations can be enabled at edge with acceptable communication cost; and 3) under the HFL framework, making the training data at edge aggregator closer to the uniform distribution is of paramount importance, which can not only improve the learning performance but also reduce the cloud communication rounds required to reach the target model accuracy. Inspired by these insights, in what follows, we cast our communication cost minimization problem in a specific HFL scenario, and achieve a satisfying trade-off between communication cost and learning performance by considering the training data distribution at edge aggregator.

III. System Description and Problem Definition

A. Hierarchical Federated Learning System

We consider a distributed federated learning system located within a broad area, where there are one cloud server and a set of N distributed computing nodes (denoted by $\mathcal{N}$). Each distributed node has imbalanced training data stored locally, which is practical in real-world systems since the underlying data is generally location- and device-related. A subset of distributed computing nodes in $\mathcal{N}$ can be selected as edge aggregators to conduct model aggregation if necessary, denoted by $\mathcal{N}_e \subset \mathcal{N}$. As shown in Fig. 4, distributed computing nodes, edge aggregators, and the cloud server collectively constitute a hierarchical federated learning system. In specific, during each round of cloud aggregation, the edge aggregators first download the global learning model from the cloud server. Then, each distributed computing node downloads the model from its associated edge aggregator, and trains the model with its own data samples. After κ_e local training iterations, each node will commit the local model updates to its associated edge aggregator. Edge aggregators conduct the model aggregations and send back the aggregated model to associated nodes. After κ_c edge aggregations, the edge aggregators will commit edge model updates to the cloud server for global model aggregations. After that, the cloud aggregator will send back the updated global model to all the edge aggregators for the next round of model training. The process repeats until a target learning accuracy is reached.

B. Problem Definition

In the HFL framework, we aim to minimize the communication cost raised by edge and cloud aggregations to reach a target learning accuracy. Specifically, we cast our communication cost minimization problem as follows.

Definition 1 (Communication Cost Minimization (CCM)). *Given a set of distributed computing nodes $\mathcal{N}$ and the set of candidate edge aggregators $\mathcal{N}_e$, how to determine a subset of edge aggregators and the node-to-edge associations, such that the total communication cost between the distributed computing nodes and the edge aggregators, and between the edge aggregators and the cloud aggregator, is minimized during the distributed learning (stopped when reaching a target learning accuracy)?*

Define $x_e \in \{0,1\}$ as a binary variable that indicates whether candidate edge node $e \in \mathcal{N}_e$ is selected as an edge aggregator ($x_e = 1$) or not ($x_e = 0$). Let the binary variable $y_{ne} \in \{0,1\}$ indicates whether node $n \in \mathcal{N}$ is associated with edge aggregator e ($y_{ne} = 1$) or not ($y_{ne} = 0$). Denote by c_{ne} the communication cost of node n committing the local model updates to its associated edge aggregator e. Define κ as the total number of cloud aggregations taken at the cloud aggregator to reach a target learning accuracy. The total amount of communication cost between distributed computing nodes and the edge aggregators can be represented as follows:

$$C_{ne}(\boldsymbol{Y}, \kappa) = \kappa\kappa_c \sum_{n\in\mathcal{N}} \sum_{e\in\mathcal{N}_e} y_{ne} c_{ne}, \tag{1}$$

where $\boldsymbol{Y} = \{y_{ne}\}_{\forall n\in\mathcal{N}, e\in\mathcal{N}_e}$ are the node-edge association results. Likewise, let c_{ec} be the communication cost of edge aggregator e committing the edge model updates to the cloud aggregator. The total amount of communication cost between

edge aggregators and the cloud aggregator can be represented as follows:

$$C_{ec}(\boldsymbol{X},\kappa)=\kappa\sum_{e\in\mathcal{N}_e}x_e c_{ec}, \tag{2}$$

where $\boldsymbol{X}=\{x_e\}_{\forall e\in\mathcal{N}}$ are the edge aggregator selection results. Then, the **CCM** problem can be formally formulated as

$$\min_{\boldsymbol{X},\boldsymbol{Y},\kappa}\ C_{ne}(\boldsymbol{Y},\kappa)+C_{ec}(\boldsymbol{X},\kappa), \tag{3}$$

$$s.t.\ \ x_e=0, \qquad \forall e\notin\mathcal{N}_e, \tag{4}$$

$$\sum_{e\in\mathcal{N}_e}y_{ne}=1, \qquad \forall n\in\mathcal{N}, \tag{5}$$

$$y_{ne}\le x_e, \qquad \forall n\in\mathcal{N},\forall e\in\mathcal{N}_e, \tag{6}$$

$$\sum_{n\in\mathcal{N}}y_{ne}\le B_e, \qquad \forall e\in\mathcal{N}_e, \tag{7}$$

$$x_e\in\{0,1\}, \qquad \forall e\in\mathcal{N}, \tag{8}$$

$$y_{ne}\in\{0,1\}, \qquad \forall n\in\mathcal{N},\forall e\in\mathcal{N}_e. \tag{9}$$

The constraint (4) means that we cannot select nodes that cannot play the role of edge aggregator as edge aggregators. The constraint (5) represents that each node has to be associated with one and only one edge aggregator. The constraint (6) requires that the candidate edge node e must be selected as an edge aggregator in order to support the association from node n. Edge devices usually have limited communication and computation resources, so we limit the number of nodes that can associate to edge aggregator e within B_e, shown in constraint (7).

C. Problem Analysis

The formulated **CCM** problem is intractable directly due to the following reasons. To optimize the problem, on the one hand, we have to make decisions on $\boldsymbol{X}$ and $\boldsymbol{Y}$ to minimize the communication cost in each round of cloud aggregation. On the other hand, we need to reduce the total number of required cloud aggregations κ, which however is unknown ahead. Moreover, the decisions on $\boldsymbol{X}$ and $\boldsymbol{Y}$ can affect the value of κ intangibly, making the problem further intricate. Based on the found insights in Section II, in what follows, we consider the data distribution at edge aggregators to pinpoint the impact of κ, based on which we then transform the original problem and devise efficient algorithms to solve it.

IV. Design of *SHARE*

In this section, we propose the optimization framework of *SHARE* to transform and solve the **CCM** problem.

A. Overview of SHARE

Figure 5 shows the workflow of *SHARE*, which includes the components of problem transformation and algorithm design. For problem transformation, as discussed above, there are two optimization directions toward solving the **CCM** problem, i.e., minimizing the communication cost in each round of cloud aggregation, and reducing the number of required cloud aggregations. We first divide the **CCM** problem into two

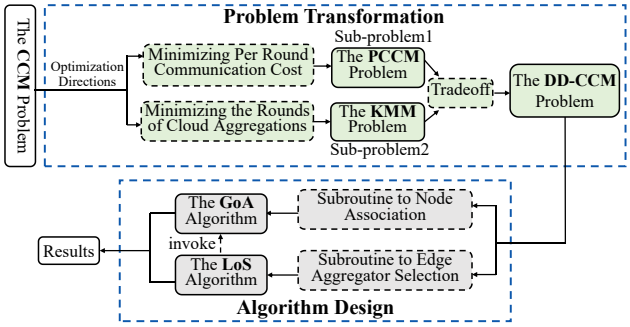


Fig. 5: The design of *SHARE*.

sub-problems corresponding to the two directions, where the sub-problem1 of per round communication cost minimization (**PCCM**) problem is formulated to minimize the per round communication cost. To pinpoint the impact of κ, we adopt the Kullback–Leibler divergence (KLD) to quantify the balance degree of data distribution at edge aggregators, based on which we then formulate the sub-problem2: KLD mean minimization (**KMM**) problem to minimize the mean of KLD values of edge aggregators, i.e., making the data distribution of edge aggregators closer to the uniform distribution. With balancing the tradeoff between two sub-problems, the original problem is then transformed to a data distribution aware communication cost minimization (**DD-CCM**) problem. Due to the NP-hardness of **DD-CCM** problem, we finally devise two light-weight algorithms to optimize the highly-coupling decisions of edge aggregator selection and distributed node association, respectively.

B. Problem Transformation

Sub-problem1 (PCCM)*. In each round of cloud aggregation, the objective is to minimize the total communication cost among distributed computing nodes, edge aggregators, and the cloud aggregator by making decisions on* $\boldsymbol{X}$ *and* $\boldsymbol{Y}$*.* That is,

$$\begin{aligned}&\min_{\boldsymbol{X},\boldsymbol{Y}}\ J_c(\boldsymbol{X},\boldsymbol{Y})=\kappa_c\sum_{n\in\mathcal{N}}\sum_{e\in\mathcal{N}_e}y_{ne}c_{ne}+\sum_{e\in\mathcal{N}_e}x_e c_{ec},\\ &s.t.\ \ constraints:(4),(5),(6),(7),(8),(9).\end{aligned} \tag{10}$$

Denote by $P_n=P(\boldsymbol{D_n})$ the data distribution of node n, and denote by $P_e=P(\cup_{\forall n\in\mathcal{M}_e}\boldsymbol{D_n})$ the data distribution of edge aggregator e, where $\mathcal{M}_e$ is the set of nodes associated to edge aggregator e. Note that, in this paper, the data distribution of each node can be unknown ahead, which can be effectively inferred from the uploaded model updates without sacrificing data privacy based on the previous work [21].

Sub-problem2 (KMM)*: The objective is to minimize the mean of KLD between the data distribution of edge aggregators and the uniform distribution by making decisions on* $\boldsymbol{X}$ *and* $\boldsymbol{Y}$*.* That is,

$$\begin{aligned}&\min_{\boldsymbol{X},\boldsymbol{Y}}\ J_d(\boldsymbol{X},\boldsymbol{Y})=\frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}}D_{KL}(P_e||P_u),\\ &s.t.\ \ constraints:(4),(5),(6),(7),(8),(9),\end{aligned} \tag{11}$$

where $\mathcal{E}$ is the set of edge aggregators (i.e., $x_e = 1, \forall e \in \mathcal{E}$), P_u is the uniform distribution, and $D_{KL}(P_e||P_u)$ is the KLD between P_e and P_u.

It should be noted that, the optimal results of *Sub-problem1* and *Sub-problem2* sometimes cannot be achieved simultaneously due to the competing objectives. The following tradeoff plays: should we associate nodes to edge aggregators in accordance with the communication cost or the data distribution at edge aggregators? To balance the tradeoff, we incorporate a parameter γ to tune the weight of communication cost and data distribution KLD. Then, the original **CCM** problem can be transformed into the following **DD-CCM** problem:

$$\begin{aligned} \min_{\boldsymbol{X},\boldsymbol{Y}} \quad & J_c(\boldsymbol{X},\boldsymbol{Y}) + \gamma J_d(\boldsymbol{X},\boldsymbol{Y}), \\ s.t. \quad & constraints : (4),(5),(6),(7),(8),(9). \end{aligned} \tag{12}$$

Challenges: To address the **DD-CCM** problem, we face the following challenges. First, we have to determine how many and which edge aggregators should be chosen, which however is non-trivial. On the one hand, if we choose more edge aggregators, the communication cost between distributed computing nodes and edge aggregators can be reduced, but the communication cost between edge aggregators and the cloud aggregator becomes heavier. On the other hand, the edge aggregators should be closer to distributed nodes to reduce the node-edge aggregator communication cost, but should also be closer to the cloud aggregator to reduce the edge-cloud communication cost, which conflicts with each other. Second, how to associate the distributed computing nodes to the edge aggregators also matters. To improve communication efficiency, we can reduce the communication cost per round by associating each node to the according closest edge aggregator. However, for learning performance, we have to consider the data distribution at edge aggregators, while they cannot be achieved simultaneously. In fact, even if we ignore the data distribution at edge aggregators, the above problem is still NP-hard, which can be a polynomial reduction from the classic NP-hard *Capacitated Facility Location Problem* [22].

C. Algorithm Design

To deal with the highly-complex coupling of edge aggregator selection and computing node association, we devise two light-weight algorithms to optimize them accordingly. Particularly, we first answer the following question: given the set of edge aggregators, how to associate distributed computing nodes to them, such that the communication cost and data distribution divergence can be jointly minimized without violating the constraints? To this end, we propose a Greedy-based nOde Association (*GoA*) algorithm to make decisions on node-to-edge aggregator associations. Based on *GoA*, we then devise a LOcal Search (*LoS*) algorithm to optimize the edge aggregator selection reversely.

Subroutine to Distributed Node Association. Given the set of edge aggregators $\mathcal{E}$, we need to determine the edge aggregator that each distributed computing node should associate to, which can be formulated as:

$$\min_{\boldsymbol{y}} \quad \kappa_c \sum_{n\in\mathcal{N}}\sum_{e\in\mathcal{E}} y_{ne}c_{ne} + \gamma\frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}} D_{KL}(P_e||P_u), \tag{13}$$

$$s.t. \quad y_{ne} = 0, \qquad \forall n \in \mathcal{N}, \forall e \notin \mathcal{E}, \tag{14}$$

$$\sum_{e\in\mathcal{E}} y_{ne} = 1, \qquad \forall n \in \mathcal{N}, \tag{15}$$

$$\sum_{n\in\mathcal{N}} y_{ne} \le B_e, \qquad \forall e \in \mathcal{E}, \tag{16}$$

$$y_{ne} \in \{0,1\}, \qquad \forall n \in \mathcal{N}, \forall e \in \mathcal{E}. \tag{17}$$

The *GoA* algorithm is devised to solve the above node association problem. As shown in Algorithm 1, we greedily associate each node to an edge aggregator that can minimize the value of the objective function (13) until all nodes are scheduled. Specifically, the algorithm traverses all unassociated nodes and available edge aggregators, and calculates the values of $\Delta J_{ne} = \kappa_c c_{ne} + \gamma\frac{1}{|\mathcal{E}|}\Delta d$. The first term of ΔJ_{ne} accounts for the communication cost between node n and edge aggregator e. The second term of ΔJ_{ne} is the average KLD reduction of associating node n with edge aggregator e, where $\Delta d \leftarrow D_{KL}(P_e + P_n||P_u) - D_{KL}(P_e||P_u))$. Based on the calculated values, we find out the association pair of the node n to the edge aggregator e when the value of ΔJ_{ne} is the smallest, and associate the node to the edge aggregator. The set of nodes that are required to be scheduled is updated, and the process repeats until all nodes are scheduled.

Algorithm 1: The workflow of *GoA* algorithm.

Input : (1) edge aggregators set $\mathcal{E}$; (2) cost c_{ne}; (3) node data distribution P_n.
Output : the node association results $\boldsymbol{Y} = \{y_{ne}\}$.

Initialize $\mathcal{M}_e \leftarrow \emptyset$, $P_e \leftarrow 0$ **for each** $e \in \mathcal{E}$;
Initialize $\mathcal{S}_n \leftarrow \mathcal{N}$;
repeat
  foreach $n \in \mathcal{S}_n$ **do**
    foreach $e \in \mathcal{E}$ **do**
      if $|\mathcal{M}_e| < B_e$ **then**
        $\Delta d \leftarrow D_{KL}(P_e+P_n||P_u)-D_{KL}(P_e||P_u))$;
        Compute $\Delta J_{ne} \leftarrow \kappa_c c_{ne} + \gamma\frac{1}{|\mathcal{E}|}\Delta d$;
      end
    end
  end
  Find the node n and the edge aggregator e with minimum ΔJ_{ne};
  $\mathcal{M}_e \leftarrow \mathcal{M}_e + \{n\}$;
  $\mathcal{S}_n \leftarrow \mathcal{S}_n - \{n\}$;
  $P_e \leftarrow P_e + P_n$;
  $y_{ne} \leftarrow 1$;
until $\mathcal{S}_n = \emptyset$
return $\{y_{ne}\}$;

Subroutine to Edge Aggregator Selection. We then focus on the edge aggregator selection problem to find the optimal edge aggregator set, during the process of which the node association decisions are achieved by adopting the *GoA*. It is easy to confirm that the edge aggregator selection problem is a combinatorial problem with $O(2^{|\mathcal{N}_e|})$ possible options. To enable a time-efficient manner, we design a local search

Algorithm 2: The workflow of *LoS* algorithm.

Initialization: Feasible solution $\mathcal{E}_s$;
repeat
  'open' operation
  foreach $e \in \mathcal{N}_e - \mathcal{E}_s$ **do**
    Compute $J(\mathcal{E}_s + \{e\})$;
    if $J(\mathcal{E}_s + \{e\}) < J(\mathcal{E}_s)$ **then**
      $\mathcal{E}_s \leftarrow \mathcal{E}_s + \{e\}$;
      break
    end
  end
  'close' operation
  foreach $e \in \mathcal{E}_s$ **do**
    Compute $J(\mathcal{E}_s - \{e\})$;
    if $J(\mathcal{E}_s - \{e\}) < J(\mathcal{E}_s)$ **then**
      $\mathcal{E}_s \leftarrow \mathcal{E}_s - \{e\}$;
      break
    end
  end
  'swap' operation
  foreach $e \in \mathcal{N}_e - \mathcal{E}_s$ **do**
    foreach $e' \in \mathcal{E}_s$ **do**
      Compute $J(\mathcal{E}_s + \{e\} - \{e'\})$;
      if $J(\mathcal{E}_s + \{e\} - \{e'\}) < J(\mathcal{E}_s)$ **then**
        $\mathcal{E}_s \leftarrow \mathcal{E}_s + \{e\} - \{e'\}$;
        break
      end
    end
  end
until *No operation can reduce the total communication cost*
return $\mathcal{E}_s$;

algorithm, i.e., *LoS*, for edge aggregator selection. Define $J_m(\mathcal{E}_s)$ as the optimal value of (13) with the given edge aggregator set $\mathcal{E}_s$. In addition, if $\mathcal{E}_s$ violates constraints (14)-(17), we set $J_m(\mathcal{E}_s) = +\infty$. Define

$$J(\mathcal{E}_s) = J_m(\mathcal{E}_s) + \sum_{e \in \mathcal{E}_s} c_{ec}, \tag{18}$$

as the optimal value of (12) with the given $\mathcal{E}_s$. We propose three types of operations to optimize the total communication cost [22], [23]. As shown in Algorithm 2, starting with a randomly selected initial feasible solution $\mathcal{E}_s$, the algorithm repeats the following three local improvement operations until no operation can reduce the total communication cost.

open (e): We randomly select a candidate edge aggregator e that is not in the current solution $\mathcal{E}_s$ and perform Algorithm 1 to calculate $J(\mathcal{E}_s + \{e\})$. If there exists a candidate aggregator e that can make $J(\mathcal{E}_s + \{e\}) < J(\mathcal{E}_s)$, we include it into the current solution $\mathcal{E}_s$.

close (e): We randomly select an edge aggregator e in the current solution $\mathcal{E}_s$ and perform Algorithm 1 to calculate $J(\mathcal{E}_s - \{e\})$. If there exists such e that can make $J(\mathcal{E}_s - \{e\}) < J(\mathcal{E}_s)$, we remove it from the current solution $\mathcal{E}_s$.

swap (e,e'): We randomly select a candidate edge aggregator e that is not in the current solution $\mathcal{E}_s$, and an edge aggregator e' in the current solution $\mathcal{E}_s$. Then, we perform Algorithm 1 to calculate $J(\mathcal{E}_s + \{e\} - \{e'\})$. If there exists such a pair of e and e' that can make $J(\mathcal{E}_s + \{e\} - \{e'\}) < J(\mathcal{E}_s)$, we include e into the current solution $\mathcal{E}_s$ and remove e' from $\mathcal{E}_s$ as well.

V. Performance Evaluation

In this section, we conduct extensive experiments to demonstrate the efficacy of *SHARE*, where the experimental setup on network topologies and learning tasks, and comparative benchmarks are well designed in detail.

A. Evaluation Methodology

Experiment Setup. We build a HFL emulation system by adopting practical learning tasks and real-world network topologies. Specifically, we adopt two widely used datasets and the corresponding models: 1) MNIST [19] dataset and a standard CNN model retrieved from PyTorch[1]; and 2) CIFAR-10 [20] dataset and the ResNet-18 model in [24]. The MNIST dataset contains a training set of $60,000$ handwritten digits samples with 10 digit labels, and the CIFAR-10 dataset consists of $50,000$ training images and $10,000$ test images with 10 classes. We consider three network topologies with different geo-locations from the Internet Topology Zoo [25], i.e., GEANT (Europe, 37 nodes), UUNET (USA, 49 nodes), and TINET (Global, 53 nodes). The selected networks have the latitude and longitude information of distributed nodes, which is used to calculate the distance among nodes via the Haversine formula [26]. We obtain the shortest path distance among nodes using the Dijkstra algorithm. In each network topology, one additional node, located in Seattle, USA, is included to represent the cloud aggregator. For data distribution at distributed nodes, we assign each node with $\frac{60,000}{|\mathcal{N}|}$ MNIST samples with one digit label. In addition, for the CIFAR-10 dataset, each node is assigned with $\frac{50,000}{|\mathcal{N}|}$ samples with 3 different sample labels.

Given the distributed node set $\mathcal{N}$, the candidate edge aggregator set $\mathcal{N}_e$ is randomly generated with a fixed size. In addition, the value of B_e for each candidate edge aggregator is randomly set within the range $[3, 10]$. For communication cost, it usually reflects by the congestion level posed to the network or the traversed distance with packets. In this paper, we mimic the communication cost by the physical distance between two communicating nodes, multiplied by a coefficient and the model size, which has been widely adopted in the literature [27]. In specific, we set $c_{ne} = 0.002 \cdot d_{ne} \cdot S_m$ and $c_{ec} = 0.02 \cdot d_{ec} \cdot S_m$, where d_{ne} (km) is the shortest path distance between node n and edge aggregator e, d_{ec} (km) is the shortest path distance between edge aggregator e and the cloud aggregator, and S_m (MB) is the model size. For the model training with MNIST and CIFAR-10 datasets, we set the size of local mini batch as $B = 32$ and the learning rate as $\eta = 0.01$.

Benchmarks. For performance comparison, the following reasonable benchmarks are designed.

- **Cloud-based FL (C-FL):** Distributed computing nodes directly communicate with the cloud aggregator to commit model updates [8].
- **Cost only CPLEX (CC):** For the HFL framework, it only takes the communication cost into consideration while

[1]Retrieved from https://github.com/pytorch/examples/blob/master/mnist.

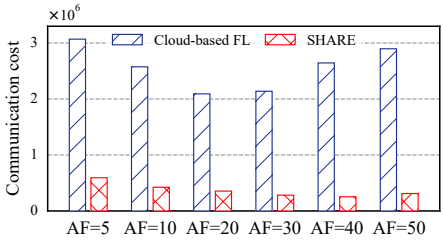


Fig. 6: *SHARE* vs. Cloud-based FL with AF $= \kappa_e \cdot \kappa_c$.

Table I: *SHARE* vs. Cloud-based FL with AF $= \kappa_e$.

$\times 10^5$	C-FL	κ_c=1	κ_c=2	κ_c=3	κ_c=4	κ_c=5	κ_c=6
AF=κ_e=5	30.70	5.93	4.23	3.74	3.55	3.32	2.82
AF=κ_e=10	25.75	5.07	3.85	2.91	3.29	2.91	2.86
AF=κ_e=20	20.92	4.04	3.36	3.30	2.65	2.21	2.84
AF=κ_e=30	21.38	4.58	3.67	3.30	3.19	2.60	2.22
AF=κ_e=40	26.44	4.56	3.30	3.11	3.08	2.50	2.37
AF=κ_e=50	28.97	4.82	4.06	3.08	3.23	2.84	2.07

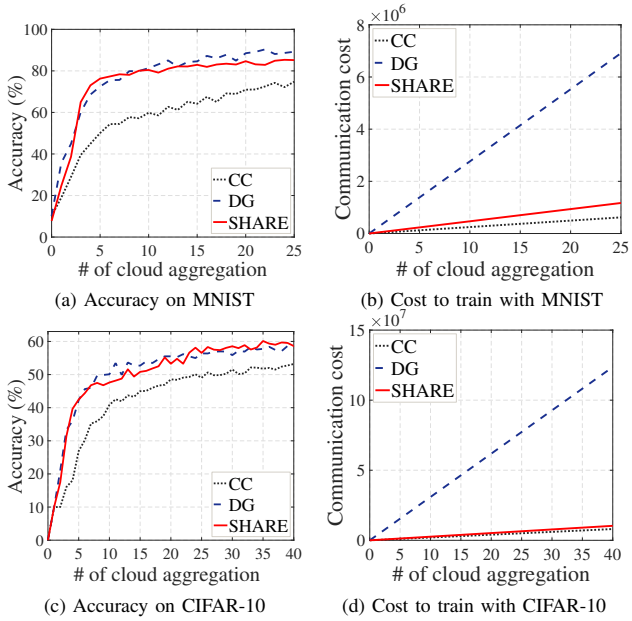
ignoring the data distribution. To solve the **DD-CCM** problem, *CC* utilizes the CPLEX mixed-integer program dynamic solver, in which the branch and cut algorithm is used to search results with the minimum per round communication cost.

- **Data only greedy (DG):** For the HFL framework, it only takes the data distribution into consideration while ignoring the communication cost among communicating nodes. To solve the **DD-CCM** problem, *DG* selects edge aggregators greedily based on the ratio of communication cost (edge aggregator to the cloud) to the node association limitation at edge aggregator, i.e., c_{ec}/B_e, until $\sum_{e\in\mathcal{E}} B_e \geq |\mathcal{N}|$. After that, *DG* utilizes the greedy strategy in [28] to associate computing nodes to edge aggregators.

B. Performance Comparison with Cloud-based FL

We first compare the performance of our proposed *SHARE* with the cloud-based FL method (i.e., *C-FL*). On the one hand, we investigate the communication costs to reach the target learning accuracy when the cloud aggregation frequency is same, i.e., AF $= \kappa_e \cdot \kappa_c$. Using the MNIST dataset and the Uunet network topology, we fix $\kappa_e =$ per 5 and vary AF from per 5 to per 50. As shown in Fig. 6, the advantage of *SHARE* in communication efficiency is significant. For example, when AF $=$ per 40 and $\kappa_e =$ per $5, \kappa_c =$ per 8, it takes 26.4×10^5 communication cost to reach the 90% learning accuracy for the approach of *C-FL*, while only 2.5×10^5 communication cost is required in *SHARE* ($\gamma = 5,000$), where more than 90% communication cost can be efficiently saved. It demonstrates that with edge aggregations, *SHARE* can reduce the cloud communication cost and improve the communication efficiency of FL significantly.

On the other hand, we set the edge aggregation frequency of *SHARE* to be the same as the cloud aggregation frequency of *C-FL*, i.e., $\kappa_e =$ AF, to explore the performance of *SHARE* under different cloud aggregation frequencies by varying the



(a) Accuracy on MNIST (b) Cost to train with MNIST

(c) Accuracy on CIFAR-10 (d) Cost to train with CIFAR-10

Fig. 7: *SHARE* vs. HFL benchmarks.

value of κ_c. Table I shows the communication cost taken by *C-FL* and *SHARE* to reach 90% MNIST accuracy under the Uunet network topology. We can have two major observations. First, seeing the case of $\kappa_c =$ per 1, we can conclude that by conducting model aggregations at edge, *SHARE* can reduce the communication cost significantly. Second, when decreasing the cloud aggregation frequency (i.e., increasing the value of κ_c), *SHARE* can further reduce the communication cost. For example, when AF $= \kappa_e =$ per 30, with κ_c increasing from per 1 to per 6, the communication cost of *SHARE* can reduce from 4.58 to 2.22. It means that with shaping data distribution at edge, *SHARE* is free from the frequent cloud model aggregations without learning performance degradation.

C. Performance Comparison with HFL Benchmarks

Then, we compare the performance of *SHARE* with the HFL benchmarks. Fig. 7 shows the performance of learning accuracy and communication cost on the MNIST and CIFAR-10 datasets, where we apply the TINET topology, and κ_e, κ_c are set to be per 5, per 40, respectively. For both learning tasks, similar observations can be achieved. First, in terms of learning accuracy and model convergence, our proposed *SHARE* ($\gamma = 25,000$ for MNIST and $\gamma = 100,000$ for CIFAR-10) is comparable to the *DG* approach with a negligible performance gap, both of which can outperform the *CC* approach significantly. Second, the communication cost of the *DG* approach is rather significant compared to our proposed *SHARE* and the *CC* approach, while *SHARE* just has a negligible cost improvement compared to that of the *CC* approach. Taking the MNIST dataset as an example, after 10 rounds of cloud aggregations, *SHARE* and *DG* can achieve an accuracy of around 80%, but only 60% accuracy can be achieved by the *CC* approach. However, for the communica-

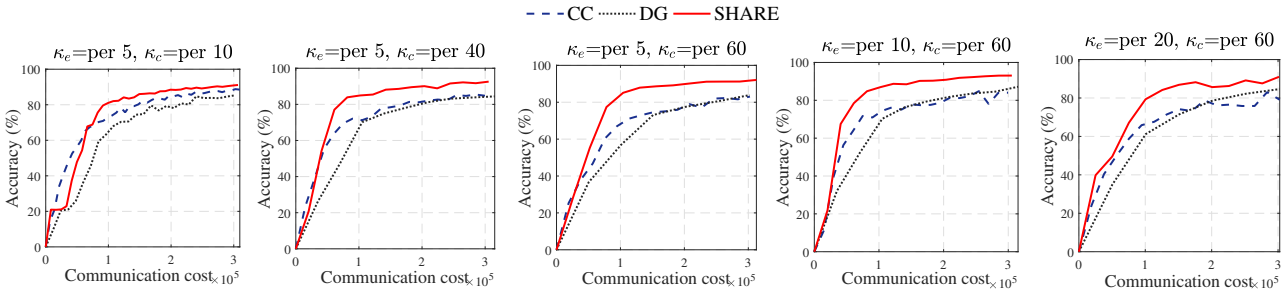
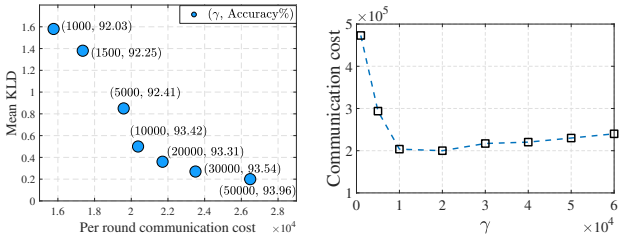


Fig. 8: The accuracy vs. communication cost under different settings of κ_e and κ_c.



(a) Tuning tradeoff (b) Required communication cost vs. γ

Fig. 9: Impact of parameter γ.

tion cost, more than 2.8×10^6 is required in the approach of *DG*, which is only 0.25×10^6 and 0.15×10^6 in the approach of *SHARE* and *CC*, respectively. It demonstrates that *SHARE* can enhance the FL performance significantly at a very small cost of communication, achieving a good balance between learning performance and communication cost.

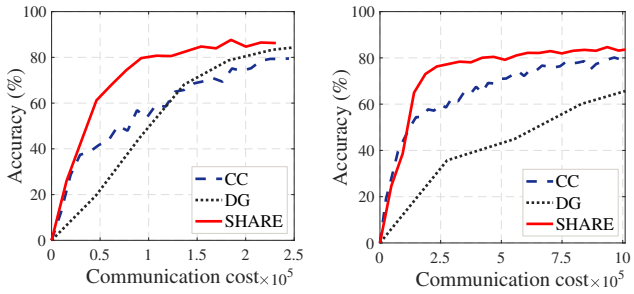
D. Impact of Parameters

With the system performance guaranteed, we then investigate the impact of system parameters, including κ_e, κ_c, and γ. Specifically, we first fix κ_e = per 5 and vary κ_c from per 10 to per 40, per 60, and then fix κ_c = per 60 with ranging κ_e from per 5 to per 10, per 20. Fig. 8 shows the performance of model accuracy and communication cost on the MNIST dataset under the GEANT network topology. We can achieve the following three observations. First, our approach can outperform the other two benchmarks in all settings. For instance, when κ_e = per 5, κ_c = per 60, the accuracy of our approach can achieve 93.18% with 3×10^5 communication cost, while the approach of *CC* and *DG* can only achieve the accuracy of 83.08% and 83.88%, respectively. In addition, 2.4×10^5 communication cost is required for the two benchmarks to reach the accuracy of 80%, while our approach only takes 0.85×10^5 communication cost to reach the accuracy, which can reduce the communication cost by 64.6%. Second, the performance gap between our approach and the benchmarks becomes significant when reducing the cloud aggregation frequency (increasing κ_c). It is reasonable since data distribution has been balanced at edge in *SHARE*, which can guarantee the learning performance without relying on frequent cloud aggregations. Third, the performance of *SHARE* can be further enhanced if the edge aggregation frequency is optimally determined. For instance, we can observe that with fixing κ_c = per 60, when reducing the edge aggregation frequency from per 5 to per 10, the learning model can converge with less communication cost, but when reducing the edge aggregation frequency to per 20, the model convergence and accuracy deteriorate. Nevertheless, the performance of *SHARE* is relatively stable and can outperform the benchmarks significantly, regardless of the parameter settings.

We then investigate the role of parameter γ to tune the tradeoff between per round communication cost and the data distribution (i.e., the mean KLD) by varying γ from $1,000$ to $50,000$. Fig. 9 shows the tradeoff results, where the model is trained with the MNIST dataset under the GEANT network topology with κ_e = per $5, \kappa_c$ = per 40. In Fig. 9 (a), we can observe that, with the increase of parameter γ, as the system cares more about the data distribution, the mean KLD is reduced while the per round communication cost is increased. Meanwhile, the model convergence accuracy is enhanced since the data distribution at edge aggregator can affect the model learning performance directly. Fig. 9 (b) shows the required communication cost when the model accuracy reaches 90%. We can observe that if a small γ is set, the required communication cost is quite significant since the impact of data distribution is not considered with priority. In addition, when the parameter is larger than a threshold (e.g., $10,000$), the potential of data distribution can be fully leveraged, and the required communication cost may increase slightly since the per round communication cost is not properly optimized. It inspires us that the impact of data distribution is significant, and the value of γ should be large enough to fully utilize the potential of shaping data distribution at edge. After achieving that, the value of γ should be controlled to consider the per round communication cost.

E. Impact of Network Topology

In this subsection, we evaluate the robustness performance of our proposed *SHARE* by checking its performance under different network topologies. Fig. 10 shows the learning performance under the network topology of UUNET and TINET, respectively, where the model is trained with the MNIST dataset with κ_e = per $5, \kappa_c$ = per 40. We can see that our approach can outperform the other two benchmarks significantly in both network topologies. For instance, in the UUNET topology, training the model to reach the learning accuracy of 80%, the communication cost of 2.4×10^5 and 2×10^5 is required in *CC* and *DG*, respectively, while our proposed *SHARE* ($\gamma = 5000$) only takes 0.9×10^5 communication cost,



(a) UUNET (b) TINET

Fig. 10: The performance under different network topologies.

improving the communication efficiency by 62.5% and 55%, respectively. In addition, under the TINET topology, to reach the accuracy of 80%, our proposed *SHARE* ($\gamma = 25,000$) can save more than 60% communication cost when compared to the *CC* approach, and has a significant communication efficiency gain compared to the *DG* approach.

VI. Related Work

Federated learning, as an emerging distributed learning technology, has attracted considerable research attention in recent years. There have been some efforts to improve the federated learning performance in terms of efficiency [29]–[32], privacy [33], [34], security [35], [36], incentives [37], [38], etc. For example, in [29], Wang *et al.* proposed a control algorithm to optimize federated learning by dynamically determining the aggregation frequency. In [34], Geyer *et al.* proposed to apply differential privacy preserving techniques to further protect user privacy in federated learning. Fang *et al.* in [36] proposed two defense mechanisms to exclude the malicious local updates for robust federated learning. Jiao *et al.* in [37] proposed an auction-based market model to incentivize more federated learning participants.

In addition to the aforementioned issues, efficient communication is also crucial for federated learning [39]. Recently, some efforts have been made to reduce the communication cost raised by federated learning, and one of the directions is to design the compression scheme for model updates. Particularly, Konečný *et al.* in [15] introduced structured and sketched model updates, and Alistarh *et al.* in [14] applied gradient quantization and encoding scheme, to reduce the data volume of mode updates. However, the communication cost is reduced at the price of learning accuracy degradation due to the incomplete model updates. Other works like [11], [17], [18], [40] attempted to reduce the required number of cloud aggregation rounds or model updates of participants. For example, Wang *et al.* in [11] proposed to dynamically exclude irrelevant individual model updates without committing them to the cloud. Ji *et al.* in [17] proposed to dynamically adjust the number of participating nodes and discard unimportant model updates. Nevertheless, in order to guarantee the learning performance, the improvement of communication efficiency is still limited. Unlike those solutions, in this paper, we explore a new direction to reduce the communication cost by migrating extensive model aggregations to edge. The advantage is that the significant communication overhead to the cloud can be avoided, while the learning performance can be guaranteed in the meantime (verified with experiments). To this end, we shed light on the HFL framework and investigate the communication efficiency issue under the framework.

Although the concept of HFL was mentioned in a few works [28], [41]–[43], the potential of HFL is not fully explored and our considered communication cost minimization problem is rarely mentioned. Specifically, Liu *et al.* in [41] proposed a collaborative training algorithm for the HFL framework. Luo *et al.* in [42] tried to achieve a resource-efficient HFL framework based on a joint computation and communication resource scheduling model. Differently, in this paper, we exploit the potential of the HFL framework via pilot experiments, and disclose that by shaping data distribution at edge, the communication efficiency of FL can be significantly enhanced. Inspired by such insight, we propose *SHARE* to optimize the communication cost raised by model aggregations in the HFL framework. To the best of our knowledge, we are the first to cast a communication-efficient HFL framework with imbalanced data at distributed computing nodes, and the design of *SHARE* can ably create a satisfying balance between communication cost and learning performance.

VII. Conclusion and Future Work

In this paper, based on pilot experimental results, we have justified the potential of using the HFL framework, where frequent model aggregations can be enabled at edge without learning performance degradation. Under the HFL framework, we have formulated the **CCM** problem to minimize the communication cost raised by edge/cloud model aggregations. Inspired by the insight that making training data at edge aggregator closer to the uniform distribution can significantly enhance the FL performance, we have proposed *SHARE* to transform and solve the **CCM** problem efficiently by shaping the data distribution at edge. Extensive experiments under various real-world network topologies with different learning tasks have been conducted, and the results have demonstrated the efficacy of *SHARE*. For our future work, we will characterize the learning quality of each node based on data analytics, and integrate the results into *SHARE* to further enhance the node association performance.

Acknowledgment

This research was supported in part by National Natural Science Foundation of China under Grants No. 62002389, 62072472, 61702562, 61702450 and U19A2067, the Key-Area Research and Development Program of Guangdong Province under Grant No. 2019B010137005, National Key R&D Program of China under Grant No. 2019YFA0706403, Natural Science Foundation of Hunan Province, China under Grant No. 2020JJ2050, 111 Project under Grant No. B18059, the Young Elite Scientists Sponsorship Program by CAST under Grant No. 2018QNRC001 and the Young Talents Plan of Hunan Province of China under Grant No. 2019RS2001.

REFERENCES

[1] Y. Huang, Y. Zeng, F. Ye, and Y. Yang, "Fair and protected profit sharing for data trading in pervasive edge computing environments," in *Proceedings of the IEEE INFOCOM'20*, 2020, pp. 1718–1727.

[2] J. Ren, D. Zhang, S. He, Y. Zhang, and T. Li, "A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet," *ACM Computing Surveys*, vol. 52, no. 6, 2019.

[3] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1117–1129, 2019.

[4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[5] F. Lyu, J. Ren, N. Cheng, P. Yang, M. Li, Y. Zhang, and X. Shen, "LEAD: Large-scale edge cache deployment based on spatio-temporal WiFi traffic statistics," *IEEE Transactions on Mobile Computing, DOI: 10.1109/TMC.2020.2984261*, pp. 1–16, Early Access, Apr. 2020.

[6] T. Pasquier, J. Singh, J. Powles, D. Eyers, M. Seltzer, and J. Bacon, "Data provenance to audit compliance with privacy policy in the internet of things," *Personal and Ubiquitous Computing*, vol. 22, no. 2, pp. 333–344, 2018.

[7] K. R. Sollins, "IoT big data security and privacy versus innovation," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1628–1635, 2019.

[8] H. B. McMahan, E. Moore, D. Ramage, S. Hampson *et al.*, "Communication-efficient learning of deep networks from decentralized data," *arXiv preprint arXiv:1602.05629*, 2016.

[9] C. Wang, Y. Yang, and P. Zhou, "Towards efficient scheduling of federated mobile devices under computational and statistical heterogeneity," *arXiv preprint arXiv:2005.12326*, 2020.

[10] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-iid data," *arXiv preprint arXiv:1806.00582*, 2018.

[11] L. Wang, W. Wang, and B. Li, "CMFL: Mitigating communication overhead for federated learning," in *Proceedings of the IEEE ICDCS'19*, 2019, pp. 954–964.

[12] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," *Advances in Neural Information Processing Systems*, vol. 31, pp. 1299–1309, 2018.

[13] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," *arXiv preprint arXiv:1704.05021*, 2017.

[14] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Advances in Neural Information Processing Systems*, 2017, pp. 1709–1720.

[15] J. Konečnỳ, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[16] D. Rothchild, A. Panda, E. Ullah, N. Ivkin, I. Stoica, V. Braverman, J. Gonzalez, and R. Arora, "FetchSGD: Communication-efficient federated learning with sketching," in *International Conference on Machine Learning*, 2020, pp. 8253–8265.

[17] S. Ji, W. Jiang, A. Walid, and X. Li, "Dynamic sampling and selective masking for communication-efficient federated learning," *arXiv preprint arXiv:2003.09603*, 2020.

[18] J. Mills, J. Hu, and G. Min, "Communication-efficient federated learning for wireless edge intelligence in IoT," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 5986–5994, 2020.

[19] Y. LeCun, "The mnist database of handwritten digits," *http://yann. lecun. com/exdb/mnist/*, 1998.

[20] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.

[21] L. Wang, S. Xu, X. Wang, and Q. Zhu, "Towards class imbalance in federated learning," *arXiv preprint arXiv:2008.06217*, 2020.

[22] M. Pal, T. Tardos, and T. Wexler, "Facility location with nonuniform hard capacities," in *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, 2001, pp. 329–338.

[23] X. Lin and S. Wang, "Joint user association and base station switching on/off for green heterogeneous cellular networks," in *Proceedings of the IEEE ICC'17*, 2017, pp. 1–6.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE CVPR'16*, June 2016.

[25] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, 2011.

[26] C. C. Robusto, "The cosine-haversine formula," *The American Mathematical Monthly*, vol. 64, no. 1, pp. 38–40, 1957.

[27] Q. Qin, K. Poularakis, G. Iosifidis, and L. Tassiulas, "SDN controller placement at the edge: Optimizing delay and overheads," in *Proceedings of the IEEE INFOCOM'18*, 2018, pp. 684–692.

[28] M. Duan, D. Liu, X. Chen, Y. Tan, J. Ren, L. Qiao, and L. Liang, "Astraea: Self-balancing federated learning for improving classification accuracy of mobile deep learning applications," in *Proceedings of the IEEE ICCD'19*, 2019, pp. 246–254.

[29] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "When edge meets learning: Adaptive control for resource-constrained distributed machine learning," in *Proceedings of the IEEE INFOCOM'18*, 2018, pp. 63–71.

[30] N. H. Tran, W. Bao, A. Zomaya, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proceedings of the IEEE INFOCOM'19*, 2019, pp. 1387–1395.

[31] L. Li, H. Xiong, Z. Guo, J. Wang, and C. Xu, "SmartPC: Hierarchical pace control in real-time federated learning system," in *Proceedings of the IEEE RTSS'19*, 2019, pp. 406–418.

[32] W. Liu, L. Chen, Y. Chen, and W. Zhang, "Accelerating federated learning via momentum gradient descent," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 8, pp. 1754–1766, 2020.

[33] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, "Beyond inferring class representatives: User-level privacy leakage from federated learning," in *Proceedings of the IEEE INFOCOM'19*, 2019, pp. 2512–2520.

[34] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," *arXiv preprint arXiv:1712.07557*, 2017.

[35] P. Blanchard, R. Guerraoui, J. Stainer *et al.*, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, 2017, pp. 119–129.

[36] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to byzantine-robust federated learning," *arXiv preprint arXiv:1911.11815*, 2019.

[37] Y. Jiao, P. Wang, D. Niyato, B. Lin, and D. I. Kim, "Toward an automated auction framework for wireless federated learning services market," *IEEE Transactions on Mobile Computing*, pp. 1–1, 2020.

[38] Y. Zhan, P. Li, Z. Qu, D. Zeng, and S. Guo, "A learning-based incentive mechanism for federated learning," *IEEE Internet of Things Journal*, 2020.

[39] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated Learning: Challenges, Methods, and Future Directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, May 2020.

[40] K. Yang, T. Jiang, Y. Shi, and Z. Ding, "Federated learning via over-the-air computation," *IEEE Transactions on Wireless Communications*, vol. 19, no. 3, pp. 2022–2035, 2020.

[41] L. Liu, J. Zhang, S. H. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proceedings of the IEEE ICC'20*, 2020, pp. 1–6.

[42] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *arXiv preprint arXiv:2002.11343*, 2020.

[43] S. Hosseinalipour, S. S. Azam, C. G. Brinton, N. Michelusi, V. Aggarwal, D. J. Love, and H. Dai, "Multi-stage hybrid federated learning over large-scale wireless fog networks," *arXiv preprint arXiv:2007.09511*, 2020.